MINI PROJECT REPORT

ON

“TransitOps - Fleet & Logistics Management System”

SUBMITTED IN PARTIAL FULLFILLMENT OF THE REQUIREMENTS OF DEGREE OF

BACHELOR OF ENGINEERING

BY

Keerthan Poojary(T21 2401085)
Neeraj Pandey(T21 2401073)
Satyam Pandey(T21 2401074)

SUBJECT

ASD&D

SUPERVISOR

Prof. Sanobar Shaikh



DEPARTMENT OFARTIFICIAL INTELLIGENCE & DATASCIENCE
THADOMAL SHAHANI ENGINEERING COLLEGE, ADVOCATE NARI GURSAHANI MARG, 37TH RD, OFF LINKING ROAD, TPS III, BANDRA (WEST), MUMBAI, 400050 UNIVERSITY OF MUMBAI

[Academic Year: 2025–26]



THADOMAL SHAHANI ENGINEERING COLLEGE, ADVOCATE NARI GURSAHANI MARG, 37TH RD, OFF LINKING ROAD, TPS III, BANDRA (WEST), MUMBAI, 400050 A.Y. - 2025-26

Certificate

This is to certify that the Mini Project entitled “TransitOps - Fleet & Logistics Management System” is a bonafide work of Keerthan Poojary, Neeraj Pandey, Satyam Pandey submitted to the University of Mumbai in partial fulfillment of the requirement for the award of the degree of “Undergraduate” in “Artificial Intelligence and Data Science”.

_____

Prof. Sanobar Shaikh

(Supervisor)

Abstract

TransitOps is a full-stack fleet and logistics management platform designed to coordinate vehicles, drivers, trips, fuel expenses, and maintenance. It combines role-based access control, organization-level data isolation, Jira work-order synchronization, real-time monitoring, and automated cloud deployment.

TABLE OF CONTENTS

Content			Page No.
Abstract			**i**
List of Figures			**ii**
1.	**Introduction**	**7**	
1.1	Background	7	
1.2	Problem Statement	7	
2.	**Implementation**	**14**	
2.1	Technologies Used	14	
2.2	Code Structure	15	
3.	**Results and Discussion**	**16**	
3.1	Result		
3.2	Discussion		
4.	**Conclusion**	**22**	

1 Introduction

1.1 Background

Modern fleet operations involve several connected activities: maintaining an accurate vehicle inventory, checking driver eligibility, assigning trips, recording fuel and other expenses, and scheduling maintenance. When these activities are handled in disconnected spreadsheets or separate tools, information can become inconsistent. A vehicle may be assigned while it is undergoing maintenance, cargo may exceed the vehicle's payload limit, or the cost of operating a trip may be difficult to calculate.

TransitOps is designed as a centralized web-based fleet and logistics management system. It brings operational records into one application and models the relationships between companies, users, roles, vehicles, drivers, trips, fuel logs, expenses, and maintenance work orders. The database uses PostgreSQL, with Prisma ORM providing data access and schema management. The frontend uses Next.js and React, while the backend exposes API endpoints through Express.js and TypeScript.

The project also connects operational work to supporting engineering tools. Atlassian Jira synchronization links maintenance records to workshop tasks, while Prometheus and Grafana provide visibility into backend performance. Docker Compose, a Caddy reverse proxy, AWS EC2 provisioning, and GitHub Actions are included to support repeatable deployment and delivery.

1.2 Problem Statement

The primary problem addressed by TransitOps is the fragmentation of fleet-related information and workflows. Fleet managers need to know which vehicles are available, which drivers can be assigned, whether a vehicle can safely carry a given load, how much a trip costs, and whether outstanding maintenance prevents a vehicle from returning to service.

The system therefore aims to provide the following capabilities:

1. Maintain vehicle and driver records, including lifecycle status, payload capacity, license expiry, safety score, and trip history.

2. Support trip creation and dispatch using source and destination coordinates, and validate cargo weight against the selected vehicle's capacity.

3. Record fuel consumption and operational expenses, enabling fuel-economy calculations and trip-linked cost analysis.

4. Track maintenance work orders and synchronize them with Jira so that issue resolution can update the corresponding maintenance status.

5. Enforce role-based permissions and company-level data isolation so that users access only the resources permitted for their role and organization.

6. Expose operational metrics and provide a repeatable deployment process for the application and its monitoring stack.

Chapter 2

Implementation

2.1 Technologies Used:

Frontend — Next.js 19 and React 19 provide the web application and routing structure. Tailwind CSS supports interface styling, and Leaflet is used for interactive maps and trip locations.

Backend — Node.js 22, Express.js, and TypeScript implement API routes and application logic. Centralized validation, authentication, authorization, and error handling help keep request processing consistent.

Database and ORM — PostgreSQL 15 stores relational records, while Prisma ORM defines the data model, manages migrations, and provides typed database access.

Authentication and authorization — JSON Web Tokens (JWT) and role-based access control (RBAC) support authenticated access and role-specific permissions across fleet, drivers, trips, fuel expenses, and analytics.

External integration — Jira Cloud REST API v3 creates maintenance issues. An inbound webhook endpoint receives Jira issue updates and synchronizes work-order status. Atlassian Remote MCP v2 is configured for AI-agent access using the supported MCP transport and authentication flow.

Observability — Prometheus scrapes the backend /metrics endpoint at a configured five-second interval. Grafana dashboards visualize request volume, errors, route-level throughput, p95 latency, and process memory.

Deployment and automation — Docker and Docker Compose package services; Caddy provides reverse-proxy routing and HTTPS support; AWS EC2 hosts the production stack; and GitHub Actions automate CI checks and deployment.

2.2 Code Structure

The repository separates application code from infrastructure and automation so that each part can be developed and maintained independently.

• frontend/ — Next.js application, pages, reusable interface components, map views, and fleet-operation screens.

• backend/ — Express.js API, TypeScript business logic, middleware, authentication and RBAC, Prisma schema and migrations, and integration handlers.

• monitoring/prometheus.yml — Prometheus scrape configuration for backend metrics.

• monitoring/grafana/provisioning/ — Grafana datasource and dashboard provisioning files.

• aws/provision-ec2.sh — automated EC2 provisioning and initial host configuration.

• .github/workflows/ci.yml — dependency setup, Prisma generation, TypeScript checks, frontend build, container build, and configuration validation.

• .github/workflows/deploy.yml — deployment workflow triggered by pushes to the main branch.

• Docker Compose and Caddy configuration — definitions for running services together and routing external requests to the frontend, backend, and monitoring endpoints

Chapter 3

Results and Discussion

3.1 Result

Fleet and vehicle inventory: vehicle records track registration, model, type, payload capacity, acquisition cost, odometer information, and lifecycle states such as Available, On_Trip, In_Shop, and Retired.

Driver management: driver profiles connect to user records and can include license information, expiry warnings, safety scores, and trip-completion history.

Trip dispatching and mapping: trips capture source and destination details and can use latitude/longitude coordinates for map display. Cargo weight is checked against the selected vehicle's capacity before dispatch.

Fuel and expense tracking: fuel logs support consumption and economy calculations in kilometres per litre (km/L). Tolls, parking, repairs, and miscellaneous expenses can be recorded and associated with trips or vehicles.

Maintenance and Jira: service types include routine service, oil change, tyre replacement, brake service, and engine repair. Logging maintenance can move a vehicle into In_Shop status and create a Jira issue. A Jira webhook can mark a work order Completed and return the vehicle to Available when the relevant issue is resolved.

Multi-tenancy and RBAC: company-level ownership supports organization isolation, while roles such as ADMIN, FLEET_MANAGER, DISPATCHER, DRIVER, SAFETY_OFFICER, and FINANCIAL_ANALYST receive permissions appropriate to their responsibilities.

Monitoring and deployment: the backend exposes a Prometheus metrics endpoint, Grafana is provisioned with a datasource and dashboard, and Docker/AWS/GitHub Actions configuration supports containerized deployment and automated verification.

The implemented system is organized around the following capabilities:

- Fleet inventory records vehicle lifecycle states: Available, On_Trip, In_Shop, and Retired; payload capacity, odometer, and acquisition cost information can be tracked.
- Driver management maintains driver records, license expiry checks, safety scores, and trip completion history.
- Trip dispatching uses map coordinates and a dispatch board, with cargo-weight validation against vehicle capacity.
- Fuel logs calculate fuel economy in km/L, while operational expenses can be associated with trips and vehicles.
- Maintenance entries can create Jira work items. Jira webhooks can update completed work orders and return vehicles to Available when issues are resolved.
- Role-based permissions cover fleet, drivers, trips, fuel/expenses, and analytics. Company-level isolation supports multiple organizations and configurable currencies and distance units.
- Prometheus collects backend metrics through /metrics, and Grafana dashboards present request volume, error rates, route-level throughput, p95 latency, and process memory.
- Docker-based deployment, Caddy routing, an EC2 provisioning script, and GitHub Actions workflows support repeatable delivery.

3.2 Discussion

The project combines day-to-day fleet workflows with integration and operational visibility. Linking maintenance records to Jira helps reduce manual status communication between fleet managers and workshop staff. Vehicle state transitions are important because they reduce the chance of dispatching a

vehicle that is currently marked as undergoing maintenance.

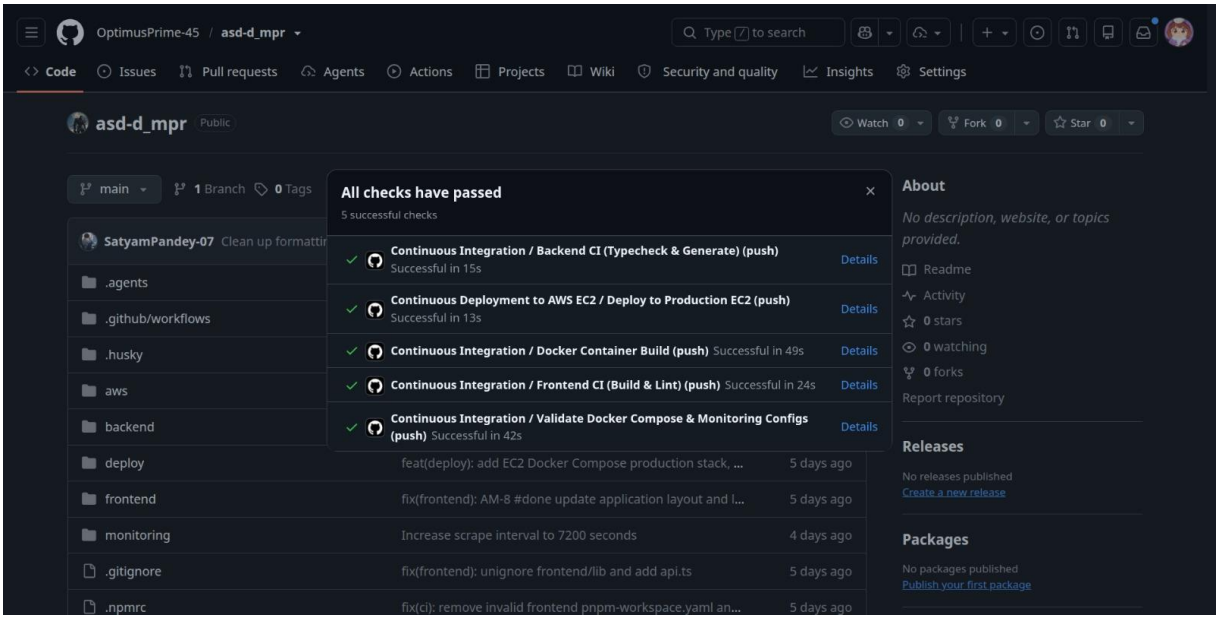
The relational schema represents the relationships between organizations, users, roles, vehicles, drivers, trips, expenses, fuel logs, and maintenance records. This makes it possible to maintain referential integrity and query operational history. Role-based permissions and company identifiers provide the foundation for separating access between user roles and organizations.

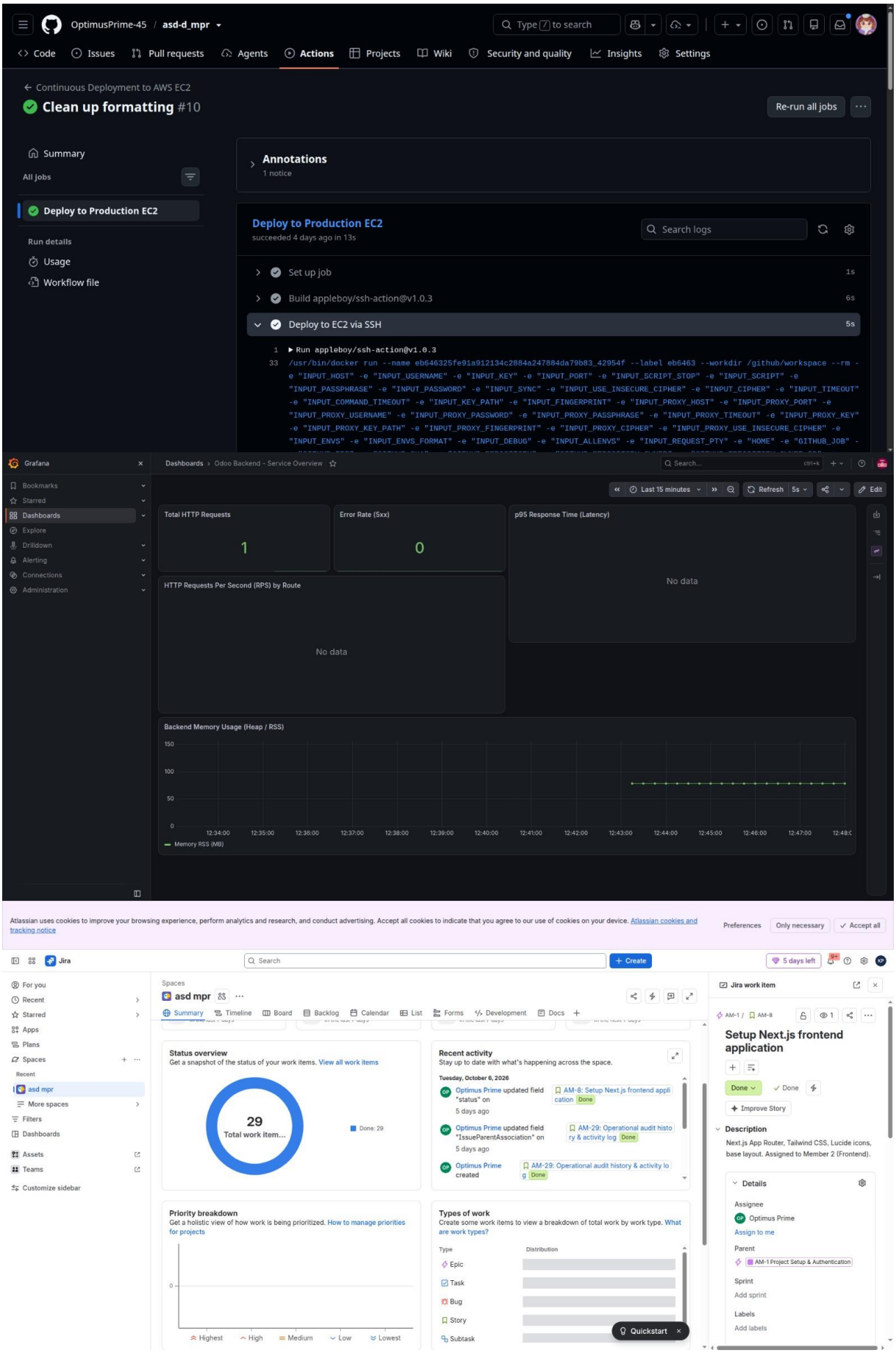
Prometheus and Grafana complement the application by making request rates, error rates, latency, and process memory visible. These measurements can help identify performance regressions, but dashboards alone do not guarantee scalability. Load testing, database-index review, backup and restore drills, alerting, and security reviews would be appropriate next steps before relying on the platform for critical production operations.

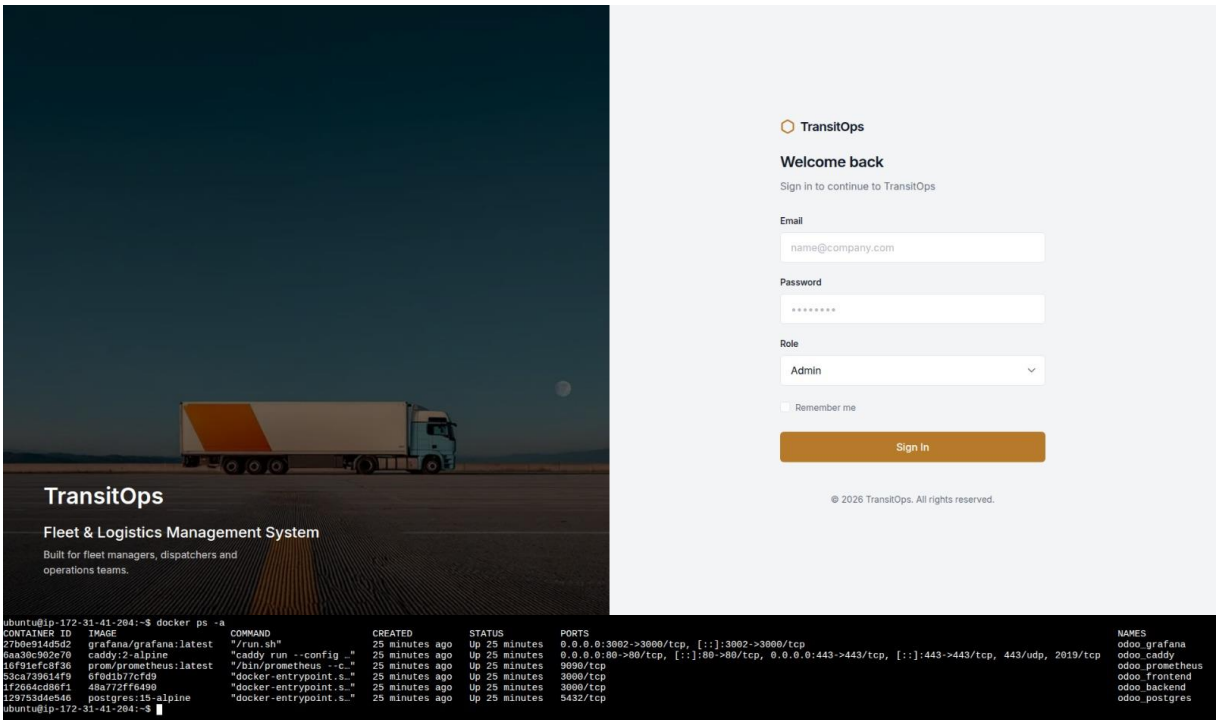
The README describes the intended architecture and deployment capabilities. Specific performance improvements or production reliability figures should only be claimed after measurements from reproducible tests; no unmeasured throughput or latency result is asserted in this report.

The design combines operational workflows with integration and monitoring features. Jira synchronization reduces the need for manual status updates between fleet managers and workshop staff. Observability helps operators inspect application performance, while multi-tenant isolation and RBAC provide boundaries for organizations and user roles. Actual production readiness still depends on secure secret management, hardened service exposure, backups, TLS configuration, and validation under realistic workloads.

SCREENSHOTS — RESERVED SPACE




OptimusPrime-45 / asd-d_mpr
Code
Issues
Pull requests
Agents
Actions
Projects
Wiki
Security and quality
Insights
Settings
Continuous Deployment to AWS EC2
Clean up formatting #10
Re-run all jobs
Summary
All jobs
Deploy to Production EC2
Run details
Usage
Workflow file
Annotations
1 notice
Deploy to Production EC2
succeeded 4 days ago in 13s
Search logs
Set up job 1s
Build appleboy/ssh-action@v1.0.3 6s
Deploy to EC2 via SSH 5s
Run appleboy/ssh-action@v1.0.3
/usr/bin/docker run --name eb646325fe91a912134c2884a247884da79b83_42954f --label eb6463 --workdir /github/workspace --rm -e "INPUT_HOST" -e "INPUT_USERNAME" -e "INPUT_KEY" -e "INPUT_PORT" -e "INPUT_SCRIPT_STOP" -e "INPUT_SCRIPT" -e "INPUT_PASSPHRASE" -e "INPUT_PASSWORD" -e "INPUT_SYNC" -e "INPUT_USE_INSECURE_CIPHER" -e "INPUT_CIPHER" -e "INPUT_TIMEOUT" -e "INPUT_COMMAND_TIMEOUT" -e "INPUT_KEY_PATH" -e "INPUT_FINGERPRINT" -e "INPUT_PROXY_HOST" -e "INPUT_PROXY_PORT" -e "INPUT_PROXY_USERNAME" -e "INPUT_PROXY_PASSWORD" -e "INPUT_PROXY_PASSPHRASE" -e "INPUT_PROXY_TIMEOUT" -e "INPUT_PROXY_KEY" -e "INPUT_PROXY_KEY_PATH" -e "INPUT_PROXY_FINGERPRINT" -e "INPUT_PROXY_CIPHER" -e "INPUT_PROXY_USE_INSECURE_CIPHER" -e "INPUT_ENVS" -e "INPUT_ENVS_FORMAT" -e "INPUT_DEBUG" -e "INPUT_ALLENVS" -e "INPUT_REQUEST_PTY" -e "HOME" -e "GITHUB_JOB" -
Grafana
Dashboards › Odoo Backend - Service Overview
Search...
Bookmarks
Starred
Dashboards
Explore
Drilldown
Alerting
Connections
Administration
Last 15 minutes
Refresh 5s
Edit
Total HTTP Requests
1
Error Rate (5xx)
0
p95 Response Time (Latency)
No data
HTTP Requests Per Second (RPS) by Route
No data
Backend Memory Usage (Heap / RSS)
150
100
50
0
12:34:00
12:35:00
12:36:00
12:37:00
12:38:00
12:39:00
12:40:00
12:41:00
12:42:00
12:43:00
12:44:00
12:45:00
12:46:00
12:47:00
Memory RSS (MB)
Atlassian uses cookies to improve your browsing experience, perform analytics and research, and conduct advertising. Accept all cookies to indicate that you agree to our use of cookies on your device. Atlassian cookies and tracking notice
Preferences
Only necessary
Accept all
Jira
Search
Create
5 days left
For you
Recent
Starred
Apps
Plans
Spaces
Recent
asd mpr
More spaces
Filters
Dashboards
Assets
Teams
Customize sidebar
Spaces
asd mpr
Summary
Timeline
Board
Backlog
Calendar
List
Forms
Development
Docs
Status overview
Get a snapshot of the status of your work items. View all work items
29
Total work item...
Done: 29
Recent activity
Stay up to date with what's happening across the space.
Tuesday, October 6, 2026
Optimus Prime updated field "status" on AM-8: Setup Next.js frontend application Done
5 days ago
Optimus Prime updated field "IssueParentAssociation" on AM-29: Operational audit history & activity log Done
5 days ago
Optimus Prime created AM-29: Operational audit history & activity log Done
Priority breakdown
Get a holistic view of how work is being prioritized. How to manage priorities for projects
Highest
High
Medium
Low
Lowest
Types of work
Create some work items to view a breakdown of total work by work type. What are work types?
Type
Distribution
Epic
Task
Bug
Story
Subtask
Quickstart
Jira work item
AM-1 / AM-8
Setup Next.js frontend application
Done
Improve Story
Description
Next.js App Router, Tailwind CSS, Lucide icons, base layout. Assigned to Member 2 (Frontend).
Details
Assignee
Optimus Prime
Assign to me
Parent
AM-1 Project Setup & Authentication
Sprint
Add sprint
Labels
Add labels


TransitOps
Fleet & Logistics Management System
Built for fleet managers, dispatchers and operations teams.
TransitOps
Welcome back
Sign in to continue to TransitOps
Email
name@company.com
Password
Role
Admin
Remember me
Sign In
© 2026 TransitOps. All rights reserved.
ubuntu@ip-172-31-41-204:~$ docker ps -a
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
27b8e914d5d2 grafana/grafana:latest "/run.sh" 25 minutes ago Up 25 minutes 0.0.0.0:3002->3000/tcp, [::]:3002->3000/tcp odoo_grafana
6aa30c902e70 caddy:2-alpine "caddy run --config …" 25 minutes ago Up 25 minutes 0.0.0.0:80->80/tcp, [::]:80->80/tcp, 0.0.0.0:443->443/tcp, [::]:443->443/tcp, 443/udp, 2019/tcp odoo_caddy
16f91efc8f36 prom/prometheus:latest "/bin/prometheus --c…" 25 minutes ago Up 25 minutes 9090/tcp odoo_prometheus
53ca739614f9 6f0d1b77cfd9 "docker-entrypoint.s…" 25 minutes ago Up 25 minutes 3000/tcp odoo_frontend
1f2664cd86f1 48a772ff6490 "docker-entrypoint.s…" 25 minutes ago Up 25 minutes 3000/tcp odoo_backend
129753d4e546 postgres:15-alpine "docker-entrypoint.s…" 25 minutes ago Up 25 minutes 5432/tcp odoo_postgres
ubuntu@ip-172-31-41-204:~$

Chapter 4

Conclusion

4.1 Conclusion

TransitOps brings core fleet operations into a unified platform. Vehicle and driver management, trip dispatching, expense tracking, and maintenance workflows are supported by a structured PostgreSQL database and an Express.js API. Jira integration connects maintenance records with workshop tasks, while Prometheus and Grafana provide operational visibility. Docker Compose, Caddy, AWS EC2 provisioning, and GitHub Actions support repeatable deployment and delivery.

Chapter 5

References

1. Next.js Documentation — https://nextjs.org/docs
2. Express.js Documentation — https://expressjs.com/
3. Prisma Documentation — https://www.prisma.io/docs
4. PostgreSQL Documentation — https://www.postgresql.org/docs/
5. Prometheus Documentation — https://prometheus.io/docs/
6. Grafana Documentation — https://grafana.com/docs/
7. Atlassian Jira Cloud REST API — https://developer.atlassian.com/cloud/jira/platform/rest/v3/
8. Model Context Protocol — https://modelcontextprotocol.io/
9. Docker Documentation — https://docs.docker.com/
10. AWS EC2 Documentation — https://docs.aws.amazon.com/ec2/